POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

EICTA First Project Delivery - ER and SQL

Author(s): **Caterina Giulia Scarpetta (11095784)**
Dario Antonino Santo (10733108)
Emily Tiphareth Arcos (11021351)
Matteo Bregola (11095785)

Group Number: **5**

Academic Year: 2024/2025

Contents

Contents **i**

1 Introduction **1**
1.1 Provided Text 1
1.2 Text Analysis 2

2 ER Model **5**
2.1 ER Model 5
2.2 Important Reasoning 6
2.3 Assumptions 7

3 Logical Model **9**

4 Relational Model **11**

5 SQL **13**
5.1 Tables Creation 13
5.2 Data Population 17
5.3 Queries 26
5.3.1 Query 1: "WHERE" 26
5.3.2 Query 2: "WHERE, LIMIT, LIKE" 27
5.3.3 Query 3: "WHERE, IN, Nested Query" 29
5.3.4 Query 4: "GROUP BY, 1 JOIN, AS" 31
5.3.5 Query 5: "WHERE, GROUP BY" 35
5.3.6 Query 6: "GROUP BY, HAVING, AS" 36
5.3.7 Query 7: "WHERE, GROUP BY, HAVING, AS" 38
5.3.8 Query 8: "WHERE, Nested Query, GROUP BY" 41
5.3.9 Query 9: "WHERE, GROUP BY, HAVING, 1 JOIN" 43
5.3.10 Query 10: "WHERE, GROUP BY, HAVING, 2 JOINs 45

1 Introduction

1.1. Provided Text

"The WellDressed fashion company wants to create a database about its products and exhibitions. The company hires fashion models to partake in fashion exhibitions. Each model provides their personal data, including Personal ID, name, surname, birth date, phone number, height, and dress size (XS, S, M, L, XL, etc.). Models are assigned to a fashion designer. These are described by their Personal ID, name, surname, stage name , phone number, and a brief description of their career. Fashion designers produce dresses for the models to wear in exhibitions. Each dress is worn by one model. Dresses are identified by a unique ID, size (XS, S, M, L, XL, etc.), colour, and production time. Dresses are made using fabrics described by a unique ID, price per square meter, name, and description of their properties. The amount of fabric used to produce a dress is also stored. Fabrics can be made from other fabrics. Models partake in exhibitions, described by a unique ID, duration (in minutes), title, location address, and description. Once a year, a fashion competition is held. The competition involves fashion designers submitting one of their dresses. The winning dress is marked as such. Dresses can be submitted to one competition only."

1.2. Text Analysis

Entities:
To start designing the ER model for this project, for the "WellDressed Fashion company", we first started by identifying the main entities from the text. We did so by deriving which concepts were related to significant attributes and connected together through relationships. The entities we found include:

- Fashion Model
- Fashion Designer
- Dress
- Fabric
- Exhibition
- Competition

Attributes:
Next, we associated attributes to the entities identified based on the text descriptions below:

- Fashion Model: "Each model provides their personal data, including *Personal ID, name, surname, birth date, phone number, height, and dress size (XS, S, M, L, XL, etc.)*"
- Fashion Designer: "described by their *Personal ID, name, surname, stage name , phone number, and a brief description of their career*"
- Dress: "identified by a *unique ID, size (XS, S, M, L, XL, etc.), colour, and production time*"
- Fabric: "described by a *unique ID, price per square meter, name, and description of their properties*" The attribute from the text describing the amount used was determined to be singular to the fabric <u>and</u> dress it refers to, and therefore this was added to the relationship between fabric and dress.
- Fashion Exhibition: "described by a *unique ID, duration (in minutes), title, location address, and description*"
- Fashion Competition: "Once a year, a fashion competition is held" This entity did not have any direct listed attributes, but given this sentence in the text we derived

that the year of the competition would be unique for each competition. We also decided to give this entity a title, as it would might be relevant.

ISA:
An important reasoning derived from the text came forward when noticing that Fashion Model and Fashion Designer shared some attributes (personal id, name, surname, phone number). From this, we concluded that these entities could be connected through an ISA relationship to a wider concept of 'Person'. This relationship created is Partial and Exclusive: partial because people can be models or designers or other, and exclusive because the hiring process detailed in the text suggests that models are brought in to be assigned to a designer and therefore cannot be both/one themselves.

Relationships:
Once the attributes for the entities were assigned, we carried out the same task for the relationships based on the descriptions in the text. Relationships were identifiable as being verbs connecting two identified entities:

- Assigned: "Models are assigned to 1 fashion designer".
- Production: "Fashion designers produce dresses".
- Partakes in: "Models partake in exhibitions".
- Wears: "Models wear dresses".
- Participation: "The competition involves fashion designers submitting one of their dresses".
- Made of: "Dresses are made using fabric", "Fabrics can be made from other fabrics".

Keys: Moreover, we began identifying the primary keys for each entity. Most entity attribute lists had unique IDs, which we interpreted as primary keys for this project as they will uniquely identify the entity by default. As stated in the Attributes segment above, the primary key for Fashion Competition was decided to be defined by a 'year' attribute as it is described as unique in the text.

Cardinalities: The text also described specific cardinality requirements, such as:

- Each model is assigned to one fashion designer.
- Each dress being worn by one model.

- Each dress being submitted to only one competition.
- Fabrics can be made from other fabrics.
- Dresses can be submitted to one competition only.

From the strict cardinality requirements above, we were able to finalize the remaining details necessary to make logical assumptions about the remaining ones in our ER model. These will be defined in the Assumptions area of the report (Section 2.3).

2 | ER Model

2.1. ER Model

The ER model was generated from the text analysis stated in the previous chapter and its elements have been positioned to respect the flow of information determined best to represent this data. Important considerations and assumptions displayed in the model are described in following sub-chapters. *Underlined attributes represent primary keys.

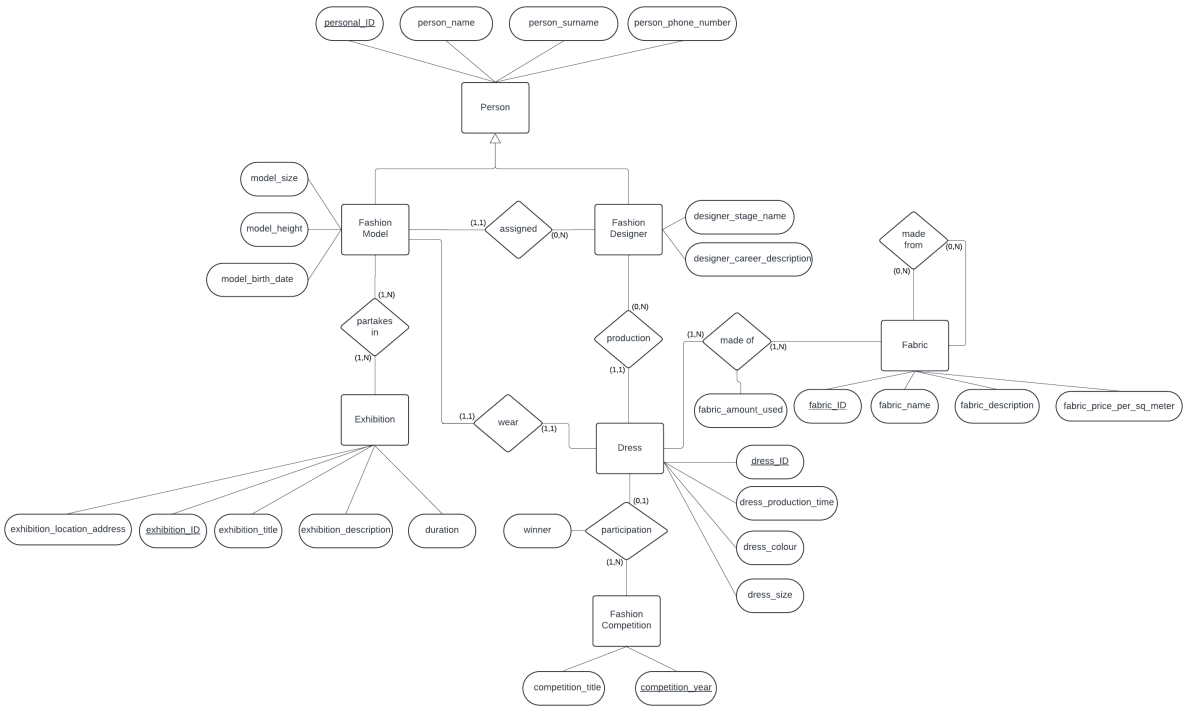

Figure 1: ER Model depicting entities, attributes, primary keys and relationships derived from WellDressed Fashion company text analysis

2.2. Important Reasoning

The ER Model was chosen to best represent the requirements detailed within the provided text. The following considerations were made to ensure this:

The team was aware that the relationships between Fashion Model, Fashion Designer and Dress create a loop which bring redundancies within the data. In fact, removing the 'assigned' relationship between Fashion Model and Fashion Designer would maintain the same information flow within the ER model (due to cardinalities) and could have been removed to avoid the loop. However, given the phrases taken from the text listed below, the team decided that removing any of listed relationships would contradict the explicit requirements of the text provided:

- *Assigned: "Models are assigned to a fashion designer."*
- *Production: "Fashion designers produce dresses for the models to wear"*
- *Wear: "Each dress is worn by one model"*

To note, the relationships between Fashion Model and Dress, and Fashion Designer and Dress (wear and production respectively) could not have been removed from the ER Model to break the loop due to their direct connections to Exhibitions and Competitions, and the same text descriptions above.

An important assumption to derive from the above decision is that data added to the database by the WellDressed fashion company must be consistent in this model - Models assigned to Fashion designers can only wear dresses produced by them and so on, and the database inputs must reflect these dependencies across all tables.

2.3. Assumptions

The following assumptions have been made from the text to create the ER model and describe cardinalities and dynamics between entities:

- A fashion model can be assigned to only one fashion designer but one fashion designer can be assigned to none or many models
- A fashion designer can produce none or many dresses and one dress can only be made by one fashion designer
- A fashion model can only wear one dress in each fashion exhibition
- A fashion model can partake in one or many fashion exhibitions and one exhibition can have one or many models partaking in it
- A dress can be made of one or many fabrics and one fabric is makes at least one or many dresses
- The amount of fabric used to make a dress is related to the dress and the fabric which makes it
- A fabric can be made from zero or many other composing fabrics
- A fashion competition has been given a title, and as a competition occurs once a year the year is a competition's unique identifier
- A fashion competition can have one or many dress participate in it and one dress can participate in no competitions or a maximum of one
- The winning dress in a competition is assigned a dress and the competition it participated in

3 | Logical Model

The Logical Model depicting entities and associated attributes and keys is defined below. Attributes underlined have been chosen as primary keys for the entity, and attributes in blue are foreign keys. Composite primary keys have been underlined as one.

Person(personal_ID, person_name, person_surname, person_phone_number)

Fashion_Model(model_birth_date, model_height, model_size, **model_ID**, **designer_ID**)

Fashion_Designer(designer_stage_name, designer_career_description, **designer_ID**)

Exhibition(exhibition_ID, exhibition_title, exhibition_location_address, duration, exhibition_description)

Dress(dress_ID, dress_size, dress_colour, dress_production_time, **designer_ID**, **model_ID**)

Fabric(fabric_ID, fabric_price_per_sq_meter, fabric_name, fabric_description)

Fashion_Competition(competition_title, competition_year)

Model_Exhibition(**model_ID**, **exhibition_ID**)

Dress_Fabric(**dress_ID**, **fabric_ID**, fabric_amount_used)

Fabric_Composition(**fabric_ID**, **composite_ID**)

Dress_Competition(**dress_ID**, **competition_year**, winner)

4 | Relational Model

From the Logical Model, we have defined the final Relational Model below.

Each table includes primary keys denoted by a '*', composite primary keys are identified by '**' for clarity. Attributes which are foreign keys have been mapped with directional arrows from their original tables and are underlined in the tables referencing them.

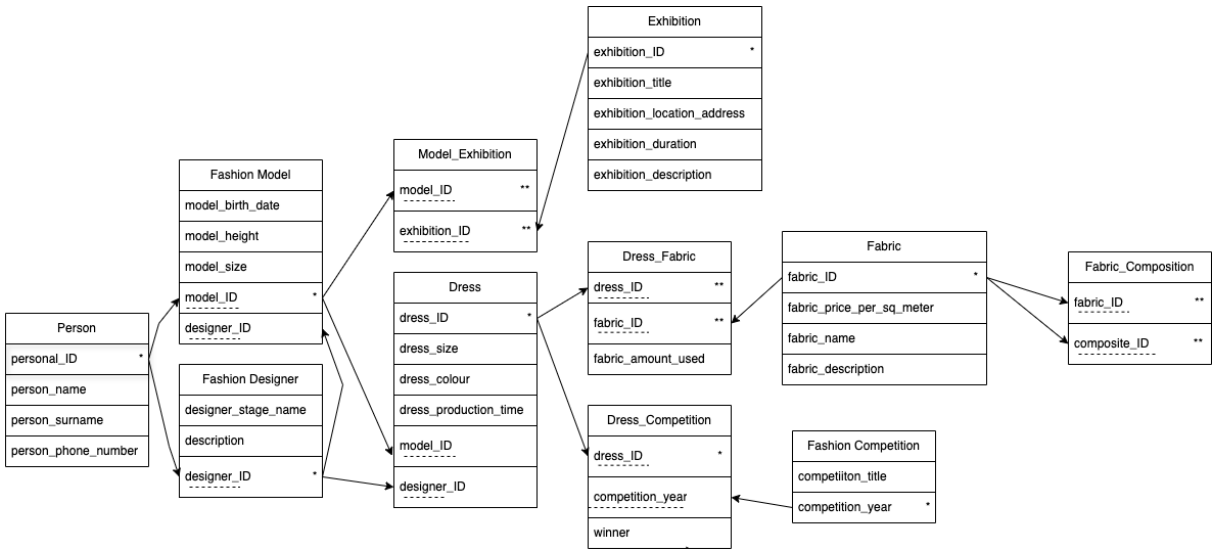

Figure 2: Relational Model depicting entities, attributes, keys and relationships from the ER model

5 | SQL

5.1. Tables Creation

```
CREATE TABLE Person(
        personal_ID VARCHAR(6) NOT NULL PRIMARY KEY,
        person_name VARCHAR(50),
        person_surname VARCHAR(50),
        person_phone_number VARCHAR(50)
);

CREATE TABLE Fashion_Model(
        model_birth_date DATE, --yyyy-mm-dd
        model_height INT, --in cm
        model_size VARCHAR(10), -- ...XS, S, M, L, ...
        model_ID VARCHAR(6),
        designer_ID VARCHAR(6),
        FOREIGN KEY (model_ID) REFERENCES Person(personal_ID),
        FOREIGN KEY (designer_ID) REFERENCES Person(personal_ID)
            ,
        PRIMARY KEY (model_ID)
);

CREATE TABLE Fashion_Designer(
        designer_stage_name VARCHAR(50),
        designer_career_description VARCHAR(500),
        designer_ID VARCHAR(6),
        FOREIGN KEY (designer_ID) REFERENCES Person(personal_ID)
            ,
        PRIMARY KEY (designer_ID)
);
```

```sql
CREATE TABLE Exhibition(
        exhibition_ID VARCHAR(6),
        exhibition_title VARCHAR(100),
        exhibition_location_address VARCHAR(50),
        duration INT UNSIGNED, --in minutes
        exhibition_description VARCHAR(500),
        PRIMARY KEY (exhibition_ID)
);

CREATE TABLE Dress(
        dress_ID VARCHAR(6),
        dress_size VARCHAR(10), -- ...XS, S, M, L, ...
        dress_colour VARCHAR(50),
        dress_production_time INT UNSIGNED, --in minutes
        model_ID VARCHAR(50),
        designer_ID VARCHAR(50),
        FOREIGN KEY (designer_ID) REFERENCES Fashion_Designer(
            designer_ID),
        FOREIGN KEY (model_ID) REFERENCES Fashion_Model(model_ID
            ),
        PRIMARY KEY (dress_ID)
);

CREATE TABLE Fabric(
        fabric_ID VARCHAR(6),
        fabric_price_per_sq_meter FLOAT(5), --assumed dollars
        fabric_name VARCHAR(50),
        fabric_description VARCHAR(500),
        PRIMARY KEY (fabric_ID)
);

CREATE TABLE Fashion_Competition(
        competition_title VARCHAR(500),
        competition_year YEAR,
        PRIMARY KEY (competition_year)
);
```

```sql
CREATE TABLE Model_Exhibition(
        -- "Partake in" Relation of the ER
        model_ID VARCHAR(6),
        exhibition_ID VARCHAR(6),
        FOREIGN KEY (model_ID) REFERENCES Fashion_Model(model_ID
            ),
        FOREIGN KEY (exhibition_ID) REFERENCES Exhibition(
            exhibition_ID),
        PRIMARY KEY(model_ID, exhibition_ID)
);

CREATE TABLE Dress_Fabric(
        -- "Made of" Relation of the ER
        dress_ID VARCHAR(6),
        fabric_ID VARCHAR(6),
        fabric_amount_used INT,
        FOREIGN KEY (dress_ID) REFERENCES Dress(dress_ID),
        FOREIGN KEY (fabric_ID) REFERENCES Fabric(fabric_ID),
        PRIMARY KEY(dress_ID, fabric_ID)
);

CREATE TABLE Fabric_Composition(
        -- "Made from" Relation of the ER
        composite_ID VARCHAR(6),
        fabric_ID VARCHAR(6),
        FOREIGN KEY (composite_ID) REFERENCES Fabric(fabric_ID),
        FOREIGN KEY (fabric_ID) REFERENCES Fabric(fabric_ID),
        PRIMARY KEY(composite_ID, fabric_ID)
);

CREATE TABLE Dress_Competition(
        -- "Participation" Relation of the ER
        dress_ID VARCHAR(6),
        competition_year YEAR,
        winner BOOL, --assigns yes or no to a unique dress and
            competition
```

```
        FOREIGN KEY (dress_ID) REFERENCES Dress(dress_ID),
        FOREIGN KEY (competition_year) REFERENCES
            Fashion_Competition(competition_year),
        PRIMARY KEY(dress_ID)
);
```

5.2. Data Population

Once the tables were created in SQL, the tool 'Mockaroo' was used to generate random data following the SQL requirements listed (data type, length etc.). Primary and foreign keys were then matched in the .csv files loaded into the database, the tables can be found below:

personal_ID	person_name	person_surname	person_phone_number
132203	Mercy	Spoors	+86 188 899 8971
139716	Minerva	Burgoin	+86 605 523 9629
146972	Adele	Capelle	+1 334 722 3351
149103	Nadine	Havelin	+55 120 328 4679
149730	Anallese	Finey	+63 931 924 9061
150064	Ruo	Slessar	+86 454 173 4983
158959	Kui	Habens	+86 201 870 0528
174186	Åslog	Jerson	+55 181 882 1341
186620	Chery	Zecchini	+880 556 181 1008
188385	Serge	Darkin	+86 771 344 0681
191180	Mahelie	Nutting	+86 175 117 8060
223350	Ye	Potter	+359 790 326 9807
242129	Mediamass	Rushford	+855 219 107 9056
243823	Rouvin	Arden	+86 744 915 6154
246042	Tania	Pecht	+231 609 520 7728
250777	Christabel	Ivakin	+84 497 319 9493
260010	Ku	Dadson	+420 212 197 3064
314069	Rachele	Kobu	+86 632 649 5241
315437	Laila	Nanetti	+84 186 351 1110
356235	Maena	Rahill	+7 975 914 2679
356312	Tades	Bothen	+7 691 804 9092
362741	Marge	Caramuscia	+62 806 634 2610
389542	Franky	Croneen	+86 267 975 8209
389547	Worthy	Creek	+7 387 731 9346
401620	Weider	Spilsburie	+86 146 947 6616
406939	Grazia	Sanney	+7 723 655 3579
434066	Severine	Frean	+33 504 308 1430
446089	Llewellyn	Theze	+420 265 148 8025
454170	Frederik	Molden	+63 361 516 2326
469547	Kalliste	Gilhooly	+420 774 258 8902
473935	Caril	Ruslinge	+1 826 720 6554
476662	Mariam	Andreacci	+86 391 107 0273
486337	Leonore	Keegan	+55 785 848 8863
489100	Leyla	Wishart	+63 484 895 5372
505754	Anae	Tucknutt	+380 461 771 7332
525437	Armin	Gircke	+86 289 767 3782
530641	Rachel	Bittany	+86 111 921 2329
561919	Zhi	Scragg	+33 282 197 8699
582354	Shell	Boatwright	+86 526 432 8389
582655	Daphnee	Finel	+224 284 748 3554
636448	Iseabal	Tomlins	+63 718 916 4395
640017	Magdalene	Queen	+351 472 578 0644
650723	Ardelle	Pendlington	+591 176 898 5377
656820	Sybyl	Jedrychowski	+380 630 752 4348
701961	Zoe	Swettenham	+63 586 448 4840
704908	Humbert	Sargison	+48 288 790 7498
722277	Noemie	Halladey	+351 715 532 4807
732291	Wood	Guerrieri	+255 858 259 6025
736290	Lotte	Poxon	+62 518 625 4616
739950	Amerigo	Jmson	+86 409 688 0717
742814	Shir	Stribling	+63 703 844 3902
757011	Indira	Lazell	+62 378 596 8151
777697	Hobey	Thornley	+62 492 134 1485
792278	Suzi	Fernihough	+257 953 506 5552
792359	Jilly	Walklott	+351 116 335 9141
796055	Lai	Botterill	+33 627 693 0473
844141	Sophie	Cabbell	+81 625 236 7635
853345	Rebecca	Tollett	+57 867 923 3808
854567	Gib	Liverock	+33 196 238 0840
855822	Elea	Grieswood	+86 605 946 7384
866930	Solene	Blesdill	+7 548 369 4088
890741	Shay	Botfield	+55 132 388 0427
893046	Caldwell	Basson	+62 677 124 3595
898125	Anet	Mostin	+48 644 950 7799

Figure 5.1: Person populated data table

model_birth_date	nodel_heigh	model_size	model_ID	designer_ID
1990-02-02	193	XS	101010	454170
1991-02-02	193	L	101011	454170
1991-02-02	193	XL	101012	454170
1991-01-02	193	M	101013	704908
1974-03-24	190	XS	132203	454170
1992-10-02	171	4XL	139716	898125
1972-04-15	189	3XL	146972	704908
1991-08-26	186	2XL	149730	996111
1995-04-10	193	S	150064	188385
1991-02-02	172	M	158959	525437
2002-11-18	215	XS	174186	401620
1983-04-13	173	S	186620	975755
1976-09-27	151	XS	191180	732291
1973-12-30	159	L	223350	844141
2000-07-26	173	XS	242129	975755
1987-04-24	182	3XL	243823	356312
1987-10-23	182	4XL	250777	732291
1983-11-10	199	M	260010	246042
1980-08-17	173	L	314069	844141
1972-08-21	196	XL	315437	246042
1980-11-22	192	S	356235	777697
1980-03-23	177	4XL	389547	188385
1990-02-15	171	L	406939	476662
1976-06-24	214	M	434066	777697
2002-12-27	180	L	446089	844141
1991-02-28	220	M	469547	742814
1983-06-16	181	2XL	473935	525437
1988-04-01	163	XL	486337	844141
1980-08-03	151	XL	505754	356312
1988-12-11	185	4XL	530641	582354
1986-11-07	156	2XL	561919	732291
1989-01-05	162	M	582655	362741
1984-10-22	185	L	640017	389542
1984-09-04	188	XS	650723	149103
2005-04-26	190	XXS	656820	362741
1998-02-08	168	S	701961	844141
1990-10-16	161	XS	722277	636448
1999-03-16	190	L	739950	792278
1980-10-07	180	4XL	757011	389542
2000-05-08	188	L	792359	777697
1990-01-29	152	L	796055	704908
1985-03-02	182	2XL	853345	898125
1985-09-13	174	S	854567	489100
1971-04-27	200	XL	855822	777697
2002-08-29	167	S	866930	792278
2002-11-07	175	3XL	890741	736290
2001-04-06	181	L	902843	742814
1990-11-12	175	L	903885	246042
1995-11-27	175	S	904025	893046
1979-06-19	182	3XL	906497	489100
2002-07-20	179	4XL	918737	995696
1997-08-04	173	M	939754	636448
2005-05-11	190	XXS	961518	401620
1991-01-19	154	2XL	968451	149103

Figure 5.2: Fashion Model populated data table

designer_stage_name	designer_career_description	designer_ID
Giorgio Armani	artisanal and handcrafted	149103
Jeremy Scott	playful and whimsical	188385
Zac Posen	urban and streetwear	246042
Ralph Lauren	avant-garde and experimental	356312
Valentino Garavani	sporty and casual	362741
Raf Simons	feminine and romantic	389542
Elliot Alderson	futuristic and tech	389547
Coco Chanel	urban and streetwear	401620
Betsey Johnson	feminine and romantic	454170
Karl Lagerfeld	vintage-inspired and retro	476662
Thom Browne	athletic and functional	489100
Calvin Klein	minimalist and monochromatic	525437
Prabal Gurung	ethereal and dreamy	582354
Isabel Marant	ethereal and dreamy	636448
Phillip Lim	ethereal and dreamy	704908
Stella McCartney	ethereal and dreamy	732291
Marc Jacobs	artisanal and handcrafted	736290
Oscar de la Renta	modern and edgy	742814
Tom Ford	sporty and casual	777697
Donatella Versace	sporty and casual	792278
Diane von Furstenberg	sleek and futuristic	844141
Alexander Wang	vintage-inspired and retro	893046
Vera Wang	timeless and classic	898125
Christian Dior	urban and streetwear	975755
Michael Kors	bohemian and free-spirited	995696
Jason Wu	athletic and functional	996111

Figure 5.3: Fashion Designer populated data table

model_ID	exhibition_ID
904025	108492
434066	120380
792359	120380
223350	138743
486337	138743
701961	138743
906497	138743
918737	138743
314069	142175
446089	142175
739950	204479
186620	349939
242129	349939
469547	349939
582655	349939
406939	350569
855822	350569
150064	359338
389547	359338
650723	370075
158959	459422
473935	459422
139716	478625
853345	478625
356235	519734
890741	519734
191180	554061
250777	554061
561919	554061
866930	554061
656820	561746
260010	577925
315437	577925
903885	577925
146972	612198
796055	612198
243823	613780
505754	613780
902843	637827
854567	654701
174186	676029
961518	676029
968451	676029
132203	725253
640017	810366
757011	810366
530641	924868
722277	977305
939754	977305
149730	986293

Figure 5.4: Exhibition populated data table

160163	4XL	Fuscia	107	757011	389542
172816	S	Crimson	75	186620	975755
176949	2XL	Maroon	93	853345	898125
187920	3XL	Mauv	178	146972	704908
200001	L	red	100	101010	454170
200002	S	Purple	90	101011	454170
200013	M	Violet	80	101012	454170
200044	XS	Aquamarine	110	101013	704908
222953	L	Crimson	83	792359	777697
226442	XS	Purple	179	722277	636448
242985	M	Violet	155	260010	246042
263539	XL	Crimson	146	855822	777697
277699	L	Khaki	72	223350	844141
287805	XS	Khaki	89	191180	732291
311248	M	Yellow	171	158959	525437
332026	4XL	Crimson	84	389547	188385
343333	S	Pink	92	854567	489100
345853	M	Green	144	939754	636448
346262	3XL	Teal	130	243823	356312
357420	M	Violet	147	469547	742814
357624	XS	Purple	171	242129	975755
359879	3XL	Pink	95	890741	736290
421933	XXS	Green	173	961518	401620
438497	XS	Teal	136	174186	401620
476938	L	Fuscia	137	446089	844141
493337	L	Yellow	77	314069	844141
496003	XS	Mauv	77	650723	149103
501994	L	Pink	159	902843	742814
509059	XL	Indigo	105	505754	356312
518247	S	Pink	92	701961	844141
596772	L	Purple	148	640017	389542
649277	S	Maroon	70	904025	893046
653514	4XL	Maroon	132	918737	995696
655953	L	Blue	140	406939	476662
670457	M	Green	77	582655	362741
675790	S	Yellow	147	150064	188385
712970	M	Puce	179	434066	777697
722826	L	Yellow	142	796055	704908
730411	S	Crimson	139	356235	777697
751383	XL	Crimson	116	486337	844141
755109	L	Turquoise	92	739950	792278
806997	4XL	Aquamarine	154	139716	898125
812451	4XL	Teal	161	250777	732291
825876	XL	Turquoise	134	315437	246042
835583	XS	Purple	125	132203	454170
841077	2XL	Goldenrod	157	149730	996111
861692	2XL	Crimson	61	561919	732291
873635	S	Aquamarine	96	866930	792278
875370	XXS	Indigo	63	656820	362741
883120	L	Khaki	156	903885	246042
891749	4XL	Fuscia	174	530641	582354
924096	2XL	Purple	71	473935	525437
950912	3XL	Indigo	154	906497	489100
991282	2XL	Mauv	173	968451	149103

Figure 5.5: Dress populated data table

competition_title	competition_year
Fashion Week	1971
SAG Awards Red Carpet	1972
Billboard Music Awards Red Carpet	1975
London Fashion Week	1979
MTV Video Music Awards Red Carpet	1980
Project Runway	1981
iHeartRadio Music Awards Red Carpet	1982
Victoria's Secret Fashion Show	1983
MTV Video Music Awards Red Carpet	1984
Victoria's Secret Fashion Show	1985
Project Runway	1987
Milan Fashion Week	1988
Oscars Red Carpet	1991
Oscars Red Carpet	1992
Paris Fashion Week	1993
BET Awards Red Carpet	1994
Oscars Red Carpet	1995
SAG Awards Red Carpet	1998
People's Choice Awards Red Carpet	1999
SAG Awards Red Carpet	2001
Emmy Awards Red Carpet	2006
Fashion Week	2011
Sundance Film Festival Fashion	2012
Golden Globes Red Carpet	2014
Project Runway	2016
Rome Fashion Day	2021
Naples Fashion Month	2022
Naples Fashion Month	2023
Venice Fashion Month	2024

Figure 5.6: Fashion Competition populated data table

fabric_ID	fabric_price_per_sq_meter	fabric_name	fabric_description
116440	7.38	tweed	antibacterial
122168	19.34	polyester	flexible
215363	4.5	canvas	flame-retardant
228073	12.01	wool	raspy
228181	6.21	velvet	breathable
236248	6.06	rayon	durable
426102	6.34	satin	silky
450464	12.38	flannel	stain-resistant
454527	14.15	knit	textured
467608	24.46	jersey	cozy
497380	10.0	suede	quick-drying
511981	17.44	organza	shiny
560850	24.52	linen	smooth
613662	3.29	fleece	eco-friendly
660086	15.57	cashmere	waterproof
663477	1.54	nylon	lightweight
683654	23.28	denim	wrinkle-resistant
724015	18.31	chiffon	absorbent
743278	22.34	mesh	luxurious
775707	5.18	corduroy	fade-resistant
846302	10.19	cotton	soft
882989	11.06	lace	insulating
907122	5.39	tulle	matte
932536	8.1	jeans	stretchy
938686	17.32	silk	silky

Figure 5.7: Fabric populated data table

dress_ID	competition_year	winner
160163	1975	1
172816	1998	1
176949	1999	0
187920	2016	1
200001	2021	1
200002	2022	1
200013	2023	1
200044	2024	1
222953	1984	1
226442	1984	0
242985	1982	0
263539	2016	0
277699	1971	0
287805	1995	0
311248	1995	0
332026	1991	1
343333	2006	1
345853	1993	1
346262	1999	1
357420	1987	0
357624	1999	0
359879	1988	1
421933	1983	1
438497	1995	0
476938	1995	1
493337	1979	0
496003	1982	1
501994	1980	1
509059	1987	0
518247	2006	0
596772	1999	0
649277	1979	1
653514	1994	1
655953	1981	1
670457	1971	0
675790	1992	0
712970	1981	0
722826	1993	0
730411	2001	0
751383	1983	0
755109	1972	1
806997	1971	1
812451	2014	1
825876	1982	0
835583	2001	1
841077	2011	1
861692	1971	0
873635	2001	0
875370	1992	1
883120	2012	1
891749	1985	1
924096	1987	1
950912	2012	0
991282	1994	0

Figure 5.8: Dress Competition populated data table

dress_ID	fabric_ID	fabric_amount_used
160163	122168	252
172816	724015	48
176949	683654	122
187920	467608	220
200001	116440	5
200002	122168	4
200013	122168	3
200044	215363	2
222953	560850	24
222953	724015	18
226442	454527	180
242985	511981	290
263539	663477	230
277699	497380	49
287805	846302	154
287805	938686	17
311248	932536	133
332026	560850	8
332026	907122	65
343333	743278	292
345853	454527	180
346262	116440	202
357420	228073	35
357420	426102	50
357420	882989	104
357624	454527	140
357624	724015	48
359879	236248	85
421933	660086	123
438497	660086	123
476938	450464	273
493337	450464	273
496003	228181	55
501994	882989	104
509059	116440	202
518247	613662	159
596772	122168	252
649277	938686	154
653514	775707	202
655953	663477	230
670457	215363	183
675790	907122	65
712970	560850	24
712970	846302	6
722826	450464	15
722826	454527	45
722826	467608	220
722826	724015	67
722826	907122	90
730411	236248	85
751383	775707	202
755109	228073	182
806997	683654	122
812451	846302	154
825876	426102	104
835583	511981	290
841077	613662	159
861692	938686	154
873635	228073	182
873635	236248	120
873635	660086	14
875370	215363	183
883120	426102	104
891749	497380	49
924096	932536	133
950912	743278	292
991282	228181	55

Figure 5.9: Dress Fabric populated data table

composite_ID	fabric_ID
683654	228181
450464	426102
724015	426102
454527	450464
236248	454527
846302	454527
775707	467608
467608	497380
613662	497380
660086	497380
882989	511981
228073	560850
497380	613662
907122	613662
215363	663477
228181	663477
116440	724015
932536	743278
511981	846302
560850	846302
938686	846302
663477	882989
426102	907122
743278	907122
122168	938686

Figure 5.10: Fabric composition populated data table

exhibition_ID	exhibition_title	exhibition_location_address	duration	exhibition_description
108492	Tokyo Fashion Week	USA	53	Avant-Garde Fashion: Pushing Boundaries
120380	Paris Fashion Week	789 Oak Rd	32	Sustainable Style: Fashion with a Purpose
138743	Tokyo Fashion Week	Australia	112	Sustainable Style: Fashion with a Purpose
142175	Seoul Fashion Week	Everywhere Village	69	Sustainable Style: Fashion with a Purpose
204479	Paris Fashion Week	123 Main St	111	Edgy Elegance: Rock and Roll Inspired Styles
349939	Berlin Fashion Week	Somewhere City	53	Natures Influence: Earthy Tones and Textures
350569	Barcelona Fashion Week	Somewhere City	113	Sustainable Style: Fashion with a Purpose
359338	Sydney Fashion Week	Canada	117	Chic and Sleek: Effortlessly Stylish
370075	Barcelona Fashion Week	789 Oak Rd	69	Whimsical Wonderland: Playful and Fun Fashion
459422	Miami Fashion Week	Everywhere Village	105	Athleisure Allure: Fashion Meets Function
478625	Toronto Fashion Week	UK	64	Retro Revival: Nostalgic Fashion Trends
519734	Vienna Fashion Week	Nowhere Town	72	Artistic Expressions: Fashion as a Canvas
554061	Paris Fashion Week	654 Maple Court	105	Vintage Glamour: Rediscovering the Past
561746	Seoul Fashion Week	UK	40	Romantic Rendezvous: Soft and Dreamy Designs
577925	Toronto Fashion Week	USA	85	Eclectic Ensembles: Mixing Patterns and Textures
612198	Vienna Fashion Week	321 Pine Lane	111	Street Style Sensations: Urban Fashion Trends
613780	Dubai Fashion Week	Australia	51	Chic and Sleek: Effortlessly Stylish
637827	Met Gala	123 Main St	49	Couture Creations: Handcrafted Masterpieces
654701	New York Fashion Week	Everywhere Village	93	Athleisure Allure: Fashion Meets Function
676029	Sydney Fashion Week	Nowhere Town	54	Vintage Glamour: Rediscovering the Past
725253	Shanghai Fashion Week	Somewhere City	33	Tropical Paradise: Vibrant Colors and Prints
810366	Seoul Fashion Week	Somewhere City	52	Global Glamour: Cultural Fusion in Fashion
924868	Toronto Fashion Week	Anytown	33	Gothic Glam: Dark and Dramatic Looks
977305	Berlin Fashion Week	789 Oak Rd	30	Natures Influence: Earthy Tones and Textures
986293	Barcelona Fashion Week	Nowhere Town	82	Bold and Beautiful: Standout Statement Pieces

Figure 5.11: Exhibition populated data table

5.3. Queries

5.3.1. Query 1: "WHERE"

1.1 Extract the titles of Fashion Exhibitions with locations ending with "Village" and whose duration lasted less than 100 minutes

```
SELECT exhibition_title
FROM exhibition
WHERE exhibition_location_address LIKE '%Village' AND duration
    <100;
```

Query Description:
The query will firstly gather all of the data from the 'exhibition' table using the FROM statement. It will then add conditions to the table using the WHERE clause starting with retrieving only rows of data where the location address ends with the word 'Village'. This is done through the LIKE statement and the percentage sign wildcard specifying the pattern needed: zero, one or many words or letters before the word Village are to be returned by the statement. Alongside the above condition, the value in the column duration (of the exhibition) must also be lower than 100 minutes. From the results of applying the two conditions to the exhibition table, the query will then SELECT to display only the titles of these exhibitions.

Query Result:

exhibition_title
Seoul Fashion Week
New York Fashion Week

Table 5.1: Results from Query 1.1

1.2 Extract the names of fabrics with prices above 15 dollars per square meter

```
SELECT fabric_name
FROM fabric
WHERE fabric_price_per_sq_meter > 15;
```

Query Description:
The query will take data from the table fabric through the FROM statement. Then using the WHERE clause it will filter fabrics only with a fabric_per_sq_meter above 15 dollars. In the end the query returns all the fabric names satisfying the condition above.

Query Result:

fabric_name
polyester
jersey
organza
linen
cashmere
denim
chiffon
mesh
silk

Table 5.2: Results from Query 1.2

5.3.2. Query 2: "WHERE, LIMIT, LIKE"

2.1 Extract the Fastest-Produced XL-Sized Dress Made from a fabric costing less than 10 dollars per square meter

```
SELECT D.dress_ID, D.dress_size, D.dress_production_time
FROM dress AS D JOIN
(
        SELECT DISTINCT dress_id
        FROM dress_fabric AS DF JOIN fabric AS F
                ON DF.fabric_ID=F.fabric_ID
        WHERE F.fabric_price_per_sq_meter < 10
) AS DFF
ON D.dress_id=DFF.dress_ID
where D.dress_size LIKE '%XL'
ORDER BY D.dress_production_time
LIMIT 1;
```

Query Description:
The "inner" query joins the Dress_Fabric and Fabric tables based on the fabric_id to associate bridge table's rows to the information about the fabric. Then the "where" clause keeps only the rows for which the fabric's price is less than 10. It's important to highlight that since Dress_Fabric's primary key is a composition of dress_id and fabric_id we could have, after the join, multiple rows for a specific dress_id. For example, a dress can be made from multiple fabrics, some of which may be expensive, and some affordable:

(dress_id=101, fabric_id=X12, price=100)
(dress_id=101, fabric_id=X13, price=5)
(dress_id=101, fabric_id=X14, price=3)

The query will return the dress if **at least one** of the fabrics used has a price below 10 (and not all of them). The "select" clause keeps only the dress_ID while the "distinct" has the purpose of removing duplicates that can occur (*in the example, the inner query without the "distinct" would return both row 2 and 3*).

The "outer" query joins the Dress table with the one resulting from the inner query based on the dress_ID. The "where" clause filters the rows for which the dress_size ends with "XL". Then the query orders the results by production time and takes only one row (namely the one that requires less time). Finally it returns the dress_id, the dress_size, and the dress_production time.

Query Result:

dress_ID	dress_size	dress_production_time
924096	2XL	71

Table 5.3: Results from Query 2.1

2.2 Extract three fashion models by id, size, and height that have a height greater than 175cm and a size that is L or larger

```
SELECT model_ID, model_height, model_size
FROM fashion_model
WHERE model_height > 175 AND model_size LIKE '%L'
LIMIT 3;
```

Query Description:
The query looks at the fashion_model table through the FROM statement. Next, it will add two conditions using the WHERE clause that state the model must have a height value greater than 175 and a size equal or greater than L. The percent wildcard allows to retrieve, zero, one or many characters before L. Then the LIMIT operator allows to retrieve the first 3 rows from the table satisfying the above condition. Then the SELECT statement will return the columns model_ID, model_height, model_size from the final table.

Query Result:

model_ID	model_height	model_size
101011	193	L
101012	193	XL
146972	189	3XL

Table 5.4: Results from Query 2.2

5.3.3. Query 3: "WHERE, IN, Nested Query"

3.1 Extract the total amount of dresses made from any fabrics costing more than 20 per square meter

```
SELECT COUNT(dress_id) AS Total_Dresses
FROM dress_fabric
WHERE fabric_id IN
    (SELECT fabric_id
    FROM Fabric
    WHERE fabric_price_per_sq_meter >20);
```

Query Description:

The "inner" query will firstly take all data from the table fabric, and apply the condition that the data returned must have fabric price per square meter greater than 20 using the WHERE clause. The table will return the fabric id of the fabrics with this price condition per square meter.

The "outer" query will firstly gather all of the data from the table dress_fabric using the FROM statement. Next, it will add a condition with the WHERE clause that extracts only the fabric_ids in the "outer" table which are also in the "inner" table.

The query will return the amount of dresses (using the COUNT function) whose dress_id is in the table satisfying the condition on the fabric_id.

Query Result:

total_Dresses
9

Table 5.5: Results from Query 3.1

3.2 Extract the average production time of dresses that won dress competitions

```
SELECT avg(dress_production_time)
FROM dress
WHERE dress_ID IN
    (SELECT dress_ID
    FROM dress_competition
```

```
    WHERE winner = 1);
```

Query Description:
First of all the "inner" query will retrieve all data from dress_competition table. Next it will return only the dresses having the condition winner equal to True that is represented by the number one using the WHERE clause. The table will return the dress ids of dresses satisfying the above condition.

The "outer" query will firstly return all data from the table dress through the FROM statement. Then it will extract only the dress_ids returned by the "inner" query thanks to the WHERE clause.

At the end the query returns the average production time (using the AVG function) of the winning dresses.

Query Result:

avg(dress_production_time)
117.5517

Table 5.6: Results from Query 3.2

5.3.4. Query 4: "GROUP BY, 1 JOIN, AS"

4.1 Extract the names of fabrics and the total amount of other fabrics they are composed of

```
SELECT f.fabric_name, COUNT(fc.composite_id) AS
    Composing_fabrics
FROM fabric AS f LEFT JOIN fabric_composition AS fc
ON f.fabric_id = fc.fabric_id
GROUP BY f.fabric_name;
```

Query Description:
The query will firstly take the table fabric and add to it columns from the fabric_composition table where the fabric id in each matches. A LEFT JOIN statement was used for this as we do not want to exclude any fabrics which are not composed of other fabrics (pure material), and instead wish to ensure that all fabrics are being returned. Next, the created

table is grouped by fabric name, where any fabrics with the same name will be grouped together into one row. Finally, the query will return the fabric names and the total of other fabrics it is made of using the COUNT function.

Query Result:

fabric_name	Composing_fabrics
tweed	0
polyester	0
canvas	0
wool	0
velvet	1
rayon	0
satin	2
flannel	1
knit	2
jersey	1
suede	3
organza	1
linen	1
fleece	2
cashmere	0
nylon	2
denim	0
chiffon	1
mesh	1
corduroy	0
cotton	3
lace	1
tulle	2
jeans	0
silk	1

Table 5.7: Results from Query 4.1

4.2 Extract fashion designers by their stage name and the number of dresses they produced

```
SELECT x.designer_stage_name, COUNT(y.dress_id) AS Total_Dresses
FROM fashion_designer AS x LEFT JOIN dress AS y
ON x.designer_id = y.designer_id
GROUP BY x.designer_stage_name;
```

Query Description:

The query will firstly add to fashion_designer table, columns from dress table where the designer ids match. The LEFT JOIN statement is used because we also want the designers that haven't produced any dresses yet. The resulting table will group rows by designer stage name using the GROUP BY statement. In the end the query will return the designer stage name and the number of dresses they made thanks to the COUNT function.

Query Result:

designer_stage_name	Total_Dresses
Giorgio Armani	2
Jeremy Scott	2
Zac Posen	3
Ralph Lauren	2
Valentino Garavani	2
Raf Simons	2
Elliot Alderson	0
Coco Chanel	2
Betsey Johnson	4
Karl Lagerfeld	1
Thom Browne	2
Calvin Klein	2
Prabal Gurung	1
Isabel Marant	2
Phillip Lim	3
Stella McCartney	3
Marc Jacobs	1
Oscar de la Renta	2
Tom Ford	4
Donatella Versace	2
Diane von Furstenberg	5
Alexander Wang	1
Vera Wang	2
Christian Dior	2
Michael Kors	1
Jason Wu	1

Table 5.8: Results from Query 4.2

5.3.5. Query 5: "WHERE, GROUP BY"

5.1 Extract dresses which have a size small or lower (XS, etc.) and the total amount of dresses produced with each size

```
SELECT dress_size, COUNT(dress_ID) AS Total_dresses
FROM Dress
WHERE dress_size LIKE '%S'
GROUP BY dress_size;
```

Query Description:
The query begins by retrieving data from the Dress table using the FROM statement. It then applies a WHERE condition to filter this data to include dresses that are LIKE S where the percentage wildcard allows for additional characters (or none) before the S to include smaller sizes such as XXS, XS etc.
Once the data gets filtered, a GROUP BY function is applied by the dress size. This makes sure that all the dresses of each size we filtered by are aggregated together.
Finally, the query counts the number of dresses of each size using the COUNT function along with the dress ID, and the total is labeled as "Total_Dresses".

Query Result:

dress_size	Total_dresses
S	8
XS	7
XXS	2

Table 5.9: Results from Query 5.1

5.2 Extract dresses by color with a production time less than 120 minutes and count them

```
SELECT dress_colour, count(dress_id) AS number_of_dresses
FROM dress
WHERE dress_production_time < 120
GROUP BY dress_colour;
```

Query Description:
The query will firstly gather data from dress table through FROM statement. Next it will retrieve only dresses with a production time less than 120 minutes using WHERE clause. Then rows will be grouped by color thanks to the GROUP BY statement. At the end the query will return dress color and the number of dresses produced with that color using the COUNT function.

Query Result:

dress_colour	number_of_dresses
Fuscia	1
Crimson	5
Maroon	2
red	1
Purple	2
Violet	1
Aquamarine	2
Khaki	2
Pink	3
Yellow	1
Mauv	1
Indigo	2
Green	1
Turquoise	1

Table 5.10: Results from Query 5.2

5.3.6. Query 6: "GROUP BY, HAVING, AS"

6.1 Extract the total amount of exhibitions presented at the same location.

```
SELECT exhibition_location_address, COUNT(exhibition_ID) AS
 total_exhibitions
FROM exhibition
GROUP BY exhibition_location_address
HAVING COUNT(exhibition_id) > 1
```

Query Description:
The query begins by selecting data from the exhibition table using the FROM statement. Then, it proceeds to group the results by the location address using the GROUP BY function, this aggregates the exhibitions based on the locations established.
Subsequently, the query applies a HAVING condition to filter the results, ensuring that only locations with more than 1 exhibition are included in the final output. This means that the query will exclusively consider the locations that have hosted multiple exhibitions.
To finalize, this query counts the total number of exhibitions for each location using the COUNT clause, this is labeled as "Total exhibitions". The result of this, will provide a list of the locations of the exhibitions together with the number of exhibitions presented at each place, but just for those places that has more than one exhibition.

Query Result:

exhibition_location_address	total_exhibitions
USA	2
789 Oak Rd	3
Australia	2
Everywhere Village	3
123 Main St	2
Somewhere City	4
UK	2
Nowhere Town	3

Table 5.11: Results from Query 6.1

6.2 Extract exhibitions grouped by title with an average duration less than the total average duration

```
SELECT exhibition_title, AVG(duration) AS average_duration
FROM exhibition
GROUP BY exhibition_title
HAVING AVG(duration) < (SELECT AVG(duration) FROM exhibition);
```

Query Description:
First of all the query will gather data from exhibition table through the FROM statement. Next it will group rows by exhibition title using the GROUP BY statement and calculates the average duration of exhibitions for each title thanks to the AVG function. Then it will filter data showing only the exhibitions with an average duration less than the overall average duration, calculated by the "inner" query. The condition above is satisfied thanks to the HAVING clause. In the end the query will show two columns: one with the exhibition title and the other, named average_duration using AS operator, with the average duration per exhibition title.

Query Result:

exhibition_title	average_duration
Seoul Fashion Week	53.6667
Berlin Fashion Week	41.5000
Toronto Fashion Week	60.6667
Dubai Fashion Week	51.0000
Met Gala	49.0000
Shanghai Fashion Week	33.0000

Table 5.12: Results from Query 6.2

5.3.7. Query 7: "WHERE, GROUP BY, HAVING, AS"

7.1 Extract the titles of red carpet fashion competitions which occurred in the 90s more than two times

```
SELECT competition_title
AS Multiple_competitions_in_the_Nineties
FROM fashion_competition
WHERE competition_year BETWEEN 1990 AND 2000 AND
    competition_title LIKE '%red_carpet%'
GROUP BY competition_title
HAVING COUNT(competition_year) > 2;
```

Query Description:
The query firstly looks at the data from table fashion_competition using the FROM statement. Next, it adds a condition using the WHERE clause that retrieves data from the table where the year the competition was held is between the year 1990 and 2000 (in

the nineties decade). The WHERE clause also filters further to ensure a condition that the competition titles include the words 'red carpet'. The percentage wildcards before and after the term 'red carpet' ensure that none, one or more letters or words can come before or after the required phrase, to ensure all red carpet events are returned. Next, the query groups the data satisfying the above condition by their title, where competitions with the same title will be grouped together. The grouped table is then further reduced using the HAVING statement to filter the table for competition titles that have occurred more than twice using the COUNT filter. Once the table is reduced to meet this condition, the query finally returns a list of the competition titles.

Query Result:

Multiple_competitions_in_the_Nineties
Oscars Red Carpet

Table 5.13: Results from Query 7.1

7.2 Extract the stage name and number of dresses made by each fashion designer, where more than 2 of their dresses took more than 80 minutes to produce

```
SELECT fd.designer_stage_name AS Stage_Name, COUNT(dr.dress_ID)
    AS TotalDresses
FROM fashion_designer AS fd
JOIN dress AS dr ON fd.designer_ID = dr.designer_ID
WHERE dr.dress_production_time > 80
GROUP BY fd.designer_stage_name
HAVING COUNT(dr.dress_ID) > 2;
```

Query Description:

This query first uses the FROM statement to retrieve data from the fashion designer table and then uses the JOIN clause to join it with the dress table using the designer id. A WHERE clause is then added to the query to exclude dresses with production times of less than or equal to 80 minutes. This guarantees that only dresses that need more than 80 minutes to make are given consideration for additional processing. The GROUP BY clause is then used to group the filtered data by the designer stage name, aggregating each designer's dress under their distinct designer ID. The HAVING condition is then applied

to further limit the results to just designers who have created more than two outfits that satisfy the production time criteria. Lastly, a list of designer stage names and the number of dresses that took more than 80 minutes to make—labeled as "TotalDresses" are returned by the query.

Query Result:

Stage_Name	TotalDresses
Phillip Lim	3
Betsey Johnson	3
Tom Ford	4
Zac Posen	3
Diane von Furstenberg	3

Table 5.14: Results from Query 7.2

5.3.8. Query 8: "WHERE, Nested Query, GROUP BY"

8.1 Extract the ranking of fashion designer IDs who have won most fashion competitions since 1990 with dresses that required more than 80 minutes to produce.

```
SELECT fd.designer_id, COUNT(*) AS wins
FROM
(
        SELECT d.designer_id, d.dress_production_time
        FROM dress_competition AS dc JOIN dress AS d ON dc.
            dress_ID=d.dress_ID
        WHERE dc.winner = 1 AND dc.competition_year > 1990
) AS fd
WHERE fd.dress_production_time > 80
GROUP BY fd.designer_ID
ORDER BY wins DESC
```

Query Description:

The subquery joins the dress_competition and dress tables based on the dress_IDs and filters the results to retain only those rows where the dress has won the competition (winner = 1) and the competition occurred after 1990. This query produces a list of fashion designer IDs, potentially including duplicates, where each instance represents a distinct win for the designer.

The outer query further filters this data to include only those entries where the dress_production_time is greater than 80. It aggregates the results by designer ID, counting the occurrences of each designer in the filtered subquery results to determine the total number of wins. Finally, the ORDER BY clause ranks the designers based on their total wins, presenting the most successful designers at the top of the list.

Query Result:

designer_id	wins
454170	3
704908	2
188385	1
489100	1
636448	1
356312	1
844141	1
995696	1
732291	1
996111	1
246042	1

Table 5.15: Results from Query 8.1

8.2 Extract the designer ids and the amount of dresses they have made for designers whose phone number begins with +62

```sql
SELECT d.designer_id, COUNT(d.dress_id) AS dresses
FROM dress AS d
WHERE d.designer_id IN
(
        SELECT fd.designer_id
        FROM fashion_designer AS fd JOIN person
        AS p ON fd.designer_id = p.personal_id
        WHERE p.person_phone_number LIKE '+62%'
)
GROUP BY d.designer_id;
```

Query Description:
The 'inner' query first takes the table fashion_designer using the FROM statement and adds the columns of table person to it using a JOIN function and by matching its designer_id to the personal_id in table person. A condition is added to filter this table, taking only rows which have person_phone_number beginning with +62. This is done using the LIKE statement and the percentage wildcard which takes the beginning of the number +62 and allows for none, one or many other values to compose the phone number. A table including the designer ids from this filtered table is returned.

The 'outer' query first takes the dress table using the FROM statement. The WHERE clause then adds the condition to this table that only rows where the designer id is also

found in the 'inner' table are kept using the IN function. The new table is then grouped by designer_id, where any duplicate designer id values are merged (a designer can create multiple dresses and therefore this removed its duplicate ids). Finally, the query returns a table with the designer ids and the total amount of dresses they have made using the COUNT function.

Query Result:

designer_ID	dresses
362741	2
736290	1
777697	4
893046	1

Table 5.16: Results from Query 8.2

5.3.9. Query 9: "WHERE, GROUP BY, HAVING, 1 JOIN"

9.1 Extract a list of fashion designers by stage name who have created dresses of the size L with a total production time above 50 minutes.

```
SELECT fd.designer_stage_name AS Stage_name, COUNT(dr.dress_id)
AS Total_Dresses
FROM fashion_designer AS fd JOIN dress AS dr ON fd.designer_id =
dr.designer_id
WHERE dr.dress_size LIKE 'L'
GROUP BY fd.designer_stage_name
HAVING SUM(dr.dress_production_time) > 50;
```

Query Description:
The query starts selecting data from the fashion designer table using the FROM statement and joins it with the dress table with the JOIN function, this matches the fashion designers with the dresses upon the designer ID. Next, the query applies a condition to filter the results, ensuring that with the WHERE clause, it only filters the dresses that are a size 'L'. This means that only this specific size will be considered on the analysis. The filtered data is then grouped by the designer stage name with the help of the GROUP BY function, aggregating the dresses under the fashion designer's stage name. The HAVING

condition allows this filter to be included in the final output only if, the designers whose total production time for the dresses exceeds 50 minutes. Lastly, the query counts the total number of dresses for each designer using the COUNT for dresses ID, which the outcome is labeled as "TotalDresses". This results in a list of fashion designers by their stage name who have created dresses of a 'L' size with a production time greater than 50 minutes.

Stage_name	Total_Dresses
Betsey Johnson	1
Tom Ford	1
Diane von Furstenberg	3
Oscar de la Renta	1
Raf Simons	1
Karl Lagerfeld	1
Phillip Lim	1
Donatella Versace	1
Zac Posen	1

Table 5.17: Results from Query 9.1

9.2 Extract colors used by Tom Ford for at least three dresses.

```
SELECT dress_colour
FROM fashion_designer AS fd JOIN dress AS d ON d.designer_ID=fd.
    designer_ID
WHERE fd.designer_stage_name='Tom Ford'
GROUP BY dress_colour
HAVING COUNT(*)>=3
```

Query Description:
The query joins the fashion designer and the dress table based on the designer_ID. The WHERE clause keeps only the rows for which the designer_stage_name is "Tom Ford". Then the GROUP BY aggregates the results by dress_colour and the HAVING clause enforces to keep only the ones that have been used at least three times. Finally, the SELECT clause displays the colours matching the conditions.

Query Result:

dress_colour
Crimson

Table 5.18: Results from Query 9.2

5.3.10. Query 10: "WHERE, GROUP BY, HAVING, 2 JOINs

10.1 Extract a list of Fashion Designers by stage name who took on average more than 2 hours to produce a dress which has won a competition

```
SELECT DISTINCT fd.designer_stage_name AS Stage_Name
FROM  (fashion_designer AS fd JOIN dress AS dr
    ON fd.designer_id = dr.designer_id)
    RIGHT JOIN dress_competition AS dc ON dr.dress_id = dc.
        dress_id
WHERE  dc.winner = 1
GROUP BY fd.designer_stage_name, dr.dress_production_time
HAVING AVG(dr.dress_production_time) >120;
```

Query Description:
The query begins by joining three tables together to allow the correct attributes to be available as a result. The first JOIN statement takes the table fashion_designer and combines its tuples to the table dress by matching the designer ids in both tables. The second JOIN statement is a right join, meaning it will take the data from the dress_competition table, and add to it all attributes from the first combined table which have a matching dress id. The right join was chosen to ensure that data returned only included items which have participated in a competition. On this new table, a condition is added using the WHERE clause, extracting only rows where the winner attribute is equal to one (the boolean statement meaning that the dress in this tuple won a competition). Next, the table of winning dresses is grouped by designer stage name, where rows in the designer_stage_name column with the same name will be grouped together in one row. The production time is also being grouped together to gather the time for each dress. A final condition is passed over the grouped data using the HAVING statement, filtering only rows where the average dress production time value is above 120 minutes using the AVG function. Finally, all unique the designer stage name are returned using the DISTINCT function for all values from the final table. The distinct value then ensures that there will be no duplicates in the final table.

Query Result:

Stage_Name
Coco Chanel
Diane von Furstenberg
Isabel Marant
Jason Wu
Karl Lagerfeld
Michael Kors
Oscar de la Renta
Phillip Lim
Prabal Gurung
Ralph Lauren
Stella McCartney
Vera Wang
Zac Posen

Table 5.19: Results from Query 10.1

10.2 Extract dresses that are associated with at least one fabric which is used in quantities exceeding 150 units and composed of at least two other fabrics

```
SELECT DISTINCT df.dress_ID
FROM (dress_fabric AS df
JOIN Fabric AS f ON df.fabric_ID = f.fabric_ID)
        JOIN Fabric_composition AS fc ON f.fabric_ID = fc.
            fabric_ID
WHERE df.fabric_amount_used>150
GROUP BY df.dress_ID, f.fabric_ID
HAVING COUNT(fc.composite_ID) >= 2;
```

Query Description:

The query performs two joins based on the fabric ID: first, it JOINS the dress_fabric table with the Fabric table, and subsequently, it joins the resulting dataset with the Fabric_composition table. The WHERE clause filters the results to include only those entries where the fabric_amount_used exceeds 150. After performing the joins and applying the filters, the query aggregates the results by dress_ID and fabric_ID, enabling a count of the associated composite_fabric entries for each fabric. The HAVING clause then filters out any groups with fewer than two composite_fabric entries. Finally, the SELECT clause retrieves distinct dress_ID values, ensuring that dresses linked to multiple fabrics meeting the "multiple composition" requirements are displayed without duplicates.

Query Result:

dress_ID
226442
263539
287805
345853
518247
655953
812451
841077

Table 5.20: Results from Query 10.2

6 | Extra

As additional work, we have deployed an instance of an Azure MySQL Database containing the same data presented in this project. Furthermore, we have created a simple Google Collaboratory script, accessible via the link below, which connects to the Azure Database and allows execution of some of the presented queries with minimal effort.
Google Collaboratory link:
`https://colab.research.google.com/drive/1uRtAEuCo_15uyh6Od1ra1hpgjQDi8i7y`